

DEV DATA P8

PROJET FINAL INTÉGRATEUR

Exploitation des données Python dans une application Web

OBJECTIF GÉNÉRAL

Dans le projet Python précédent, vous avez :

- récupéré et analysé un fichier de données ;
- validé les informations ;
- séparé les données valides et invalides ;
- structuré les données avec Python ;
- calculé les moyennes par matière ;
- exporté les résultats au format JSON.

L'objectif de ce nouveau projet est de réutiliser ces données pour construire une application web professionnelle complète utilisant :

- Python
- FastAPI
- PostgreSQL
- SQL écrit manuellement
- HTML / CSS / JavaScript
- Chart.js
- Linux
- Git et GitHub

OBJECTIFS PÉDAGOGIQUES

Ce projet permet de consolider et d'intégrer :

- Algorithmique
- Python
- Programmation Orientée Objet (POO)
- Manipulation de fichiers JSON
- SQL et PostgreSQL
- Framework Python (FastAPI)
- API REST
- HTML / CSS / JavaScript
- Visualisation de données
- Linux et scripts Bash
- Git et GitHub

PÉRIMÈTRE DU PROJET

Phase 1 (Obligatoire)

L'application doit fonctionner exclusivement avec les données contenues dans :

- valides.json

Phase 2 (Optionnelle)

Si toutes les fonctionnalités principales sont terminées, vous pourrez intégrer :

- invalides.json

afin de permettre leur affichage, leur correction et leur transfert vers les données valides.

DONNÉES DISPONIBLES

À l'issue du projet Python, vous disposez de :

- valides.json → contient les données valides
- invalides.json → contient les données invalides (optionnel)

STRUCTURE DES DONNÉES

Chaque étudiant contient au minimum :

- code
- numéro
- prénom
- nom
- date de naissance
- classe
- matières
- notes de devoir
- note d'examen
- moyenne par matière
- moyenne générale

Chaque étudiant :

- appartient à une classe ;
- possède plusieurs matières ;
- chaque matière contient :
 - une liste de notes de devoir ;
 - une note d'examen ;
 - une moyenne calculée.

TECHNOLOGIES À UTILISER

Backend

- Python
- FastAPI
- Pydantic
- PostgreSQL
- psycopg ou psycopg2
- SQL écrit manuellement (sans ORM)

Frontend

- HTML
- CSS
- JavaScript (fetch())

Visualisation

- [Chart.js](#)

Système

- Linux
- Bash

Gestion de version

- Git
- GitHub

RÈGLES DE FONCTIONNEMENT

Source principale

PostgreSQL constitue la source principale de l'application.

Source secondaire

Le fichier valides.json constitue la source secondaire.

- Il contient les données initiales générées lors du projet Python.
- Il est utilisé uniquement pour compléter l'affichage lorsque la base de données ne contient pas suffisamment de données.
- Les données provenant du fichier JSON sont affichées en lecture seule.
- Chaque ligne doit indiquer son origine :
 - DB : PostgreSQL
 - JSON : fichier valides.json

Chargement intelligent et pagination des données

Au démarrage de l'application, les données ne sont pas toutes chargées en une seule fois. L'affichage se fait progressivement, page par page.

Chargement initial

Lors de l'ouverture de l'application :

1. Le système demande uniquement la première page.
2. Le nombre de lignes affichées par défaut est de 5.
3. Le backend tente d'abord de récupérer les données non archivées de PostgreSQL.
4. Si la base de données contient moins de 5 lignes correspondant aux critères demandés, le système complète automatiquement avec les données provenant du fichier valides.json.
5. Les données retournées peuvent donc provenir :
 - uniquement de PostgreSQL ;
 - uniquement du fichier JSON ;
 - ou d'une combinaison des deux sources.
6. Chaque ligne affichée doit indiquer clairement son origine :
 - DB : donnée provenant de PostgreSQL ;
 - JSON : donnée provenant du fichier valides.json.

Chargement des pages suivantes

Les pages suivantes ne sont chargées que lorsque l'utilisateur en fait la demande, par exemple en cliquant sur :

- Suivant
- Précédent
- un numéro de page

ou par scrolling de la page(optionnelle)

À chaque demande de page, le backend :

1. reçoit le numéro de la page souhaitée ainsi que le nombre de lignes par page ;
2. applique les critères de recherche et de filtrage éventuels ;
3. récupère les données non archivées de PostgreSQL ;
4. complète si nécessaire avec les données du fichier JSON ;
5. fusionne les résultats ;
6. sélectionne uniquement les lignes correspondant à la page demandée ;
7. retourne ces données au frontend.

Modification

Seules les données provenant de PostgreSQL peuvent être modifiées.

Ajout

Toute nouvelle donnée est enregistrée uniquement dans PostgreSQL.

Synchronisation JSON → PostgreSQL

Les données issues du fichier JSON peuvent être sélectionnées puis importées dans PostgreSQL.

- Une ou plusieurs lignes peuvent être cochées.
- Les doublons doivent être détectés.
- Le champ numero identifie un étudiant de manière unique.
- Si un étudiant avec le même numero existe déjà, la ligne n'est pas importée.
- Une fois importée, la donnée devient de type DB et peut être modifiée.

Archivage

Les données archivées :

- ne sont plus affichées dans les listes principales ;
- ne sont pas supprimées physiquement ;
- restent stockées dans PostgreSQL ;
- peuvent être restaurées ultérieurement.

FONCTIONNALITÉS PRINCIPALES

1. Affichage des données

- chargement automatique au démarrage ;
- pagination par défaut de 5 lignes ;
- choix du nombre de lignes par page ;
- navigation entre les pages ;
- affichage de l'origine (DB ou JSON).

2. Recherche et filtrage

- recherche par numéro ;
- recherche par code ;
- recherche par nom ou prénom ;
- filtrage par classe ;
- filtrage par source ;
- filtrage par validité.

3. Sélection et importation

- possibilité de cocher une ou plusieurs lignes issues du fichier JSON ;
- bouton « Importer vers la base » ;
- insertion dans PostgreSQL ;
- lignes importées ensuite modifiables.

4. Ajout

- formulaire d'ajout ;
- validation côté frontend ;
- validation côté backend ;
- insertion dans PostgreSQL.

5. Modification

Édition cellule par cellule

- double-clic sur une cellule ;
- transformation en champ éditable ;
- validation avec Entrée ;
- annulation avec Échap ;
- retour au mode texte.

Édition d'une ligne complète

- case à cocher « Mode édition » ;
- tous les champs deviennent éditables ;
- bouton de sauvegarde.

Restriction

- seules les lignes provenant de PostgreSQL sont modifiables.

6. Archivage

- bouton « Archiver » ;
- exclusion automatique des résultats ;
- consultation des archives ;
- restauration.

7. Dashboard et visualisation des données

L'application doit comporter une page de tableau de bord permettant de visualiser des statistiques et des indicateurs à partir des données stockées dans PostgreSQL et, si nécessaire, complétées par les données issues du fichier CSV.

Technologies à utiliser

- HTML
- CSS
- JavaScript
- [Chart.js](#)

Objectifs du dashboard

Le tableau de bord doit permettre de :

- afficher des indicateurs globaux (KPI)
- visualiser la répartition des données
- présenter des statistiques sous forme de graphiques
- faciliter l'analyse et l'interprétation des données

Indicateurs à afficher

L'application doit au minimum afficher :

- nombre total de données
- nombre de données provenant de la base de données
- nombre de données provenant du fichier CSV
- nombre de données valides
- nombre de données invalides
- nombre de données archivées

Statistiques à calculer

Le système doit produire au minimum les statistiques suivantes :

- répartition des données par classe
- répartition des données par source (DB / CSV)
- répartition des données valides et invalides
- moyenne générale par classe
- top 10 des meilleures moyennes